

Este guia contém as convenções de pseudocódigo adotado na disciplina. Entenda que, ao procurar o material de outros professores, algumas coisas podem diferir do que foi apresentado aqui.

1 Organização de um programa em pseudocódigo

Pseudocódigo é uma forma de descrever os passos de um algoritmo usando uma linguagem simples, que mistura a estrutura da programação com alguma linguagem natural (como o português ou o inglês).

Pense nele como um rascunho universal. Embora possua uma estrutura semelhante a de uma linguagem de programação, ela não é executável. Em contrapartida, a aproximação de sua estrutura com a de linguagens conhecidas, como Python, C, Java, faz com que a tradução de um “código” nesta linguagem rascunho para uma linguagem de programação seja facilitada. Pense no pseudocódigo como uma versão restrita da linguagem que você usa, tornando-a “mecânica” e facilmente compreensível a outros dispositivos que podem compreendê-la.

Nossos programas serão divididos em 3 regiões: declaração de funções, declaração de procedimentos e corpo principal do programa. Dentre as 3 regiões, apenas o corpo principal do programa possui um delimitador próprio. As regiões de procedimentos e funções não possuem um delimitador, mas cada função e cada procedimento possui um delimitador específico.

Podemos então ilustrar um exemplo estrutural de um programa em pseudocódigo abaixo:

```
fun <nome da função>(<parametros>):  
    <bloco de comandos da função>  
endf  
...  
fun <nome da função>(<parametros>):  
    <bloco de comandos da função>  
endf  
  
proc <nome do procedimento>(<parâmetros>):  
    <bloco de comandos do procedimento>  
endp  
...  
proc <nome do procedimento>(<parâmetros>):  
    <bloco de comandos do procedimento>  
endp  
  
INICIO  
    <Comandos do programa principal>  
FIM
```

Funções

Procedimentos

Corpo principal

Sempre que você for escrever um programa em pseudocódigo é necessário que os delimitadores INICIO e FIM sejam declarados.

2 Comandos básicos

Como discutimos em sala de aula, em todas as linguagens de programação, podemos dividir as instruções/comandos em 3 categorias: comandos básicos, desvios condicionais e estruturas de repetição. Além disso, os comandos básicos podem ser divididos em duas categorias: comandos de entrada e saída (comandos E/S) e as atribuições.

Os comandos de entrada e saída usam, respectivamente, as palavras **ler** e **exibir**. Tipicamente, um dado é lido por linha. Desta forma, escrevemos **ler(*x*)** para indicar que os dados escritos na linha foram transferidos para a variável *x*. Assim, os comandos de leitura fazem a captação das informações do teclado e armazenando estes dados na região de memória representada pela variável *x*. Caso tenhamos um número fixo de dados por uma linha, podemos ler um dado para cada variável. Suponha que tenhamos *k* dados em uma linha e queiramos ler cada um para uma variável. Neste caso, usamos a instrução **ler(*x*₁, *x*₂, ..., *x*_{*k*})**. Se desejarmos ler os dados para uma lista, escreveremos **ler(**L**[])** para indicar que os dados da linha lida serão distribuídos na lista **L**.

Para imprimir na tela as informações, usaremos o comando **exibir**. O argumento do comando pode ser uma variável, uma sequência de variáveis ou um texto que deve estar escrito entre aspas.

O comando de atribuição é realizado usando o símbolo $\leftarrow$. O literal para onde a setinha aponta é a variável que vai armazenar o dado que está à direita da setinha. O dado à direita pode ser diretamente uma informação, mas também o resultado de uma expressão aritmética, o resultado de uma expressão lógica, o retorno de uma função ou resultado de alguma outra operação. Antes de ilustrarmos a aplicação dos comandos básicos, precisamos revisar os tipos de dados disponíveis e as operações que podemos executar.

2.1 Tipos de dados

Seguiremos a ideia dos tipos de dados básicos do Python. Assim, podemos manipular dados booleanos, números inteiros, números reais, cadeias de caracteres (*strings*), listas e conjuntos. As cadeias de caracteres devem ser escritas entre aspas duplas. Para representar uma lista vazia escreveremos []. Para representar um conjunto vazio escreveremos {}.

2.2 Operações aritméticas

As operações aritméticas podem ser aplicadas a números inteiros e números reais. Podemos misturar dados inteiros e reais sem causar erros. Algumas operações, como soma, subtração, multiplicação, irão resultar em um dado do mesmo tipo dos operandos. Se ambos forem do mesmo tipo, o resultado será o mesmo. Se um for inteiro e o outro for real, o resultado final será real.

As seguintes operações matemáticas devem ser consideradas:

– : inversão de sinal;

+ : soma de dois números;

$-$: diferença de dois números;

$/$: divisão real de um número por outro (independente de os operandos serem inteiros ou reais);

$//$: divisão inteira de um número por outro (apenas para operandos inteiros);

$\times$: multiplicação de dois números;

mod : resto da divisão inteira do número da esquerda pelo número da direita (apenas para operandos inteiros).

2.3 Operações lógicas

Trabalhamos com noções de lógica bivaloradas, ou seja, que possui apenas dois valores-verdade: **True**, o verdadeiro, e **False**, o falso. Operações lógicas e operações comparativas retornam valores-verdade. As operações lógicas típicas são a negação **not**, a conjunção **and** e a disjunção **or**. A negação inverte o valor-verdade. A conjunção é verdadeira se ambos forem verdade. A disjunção é verdadeira se ao menos um é verdade. A tabela a seguir mostra o resultado das operações.

A	B	not A	A and B	A or B
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

2.4 Operações comparativas

As operações de comparação serão apenas entre números inteiros e reais. As seguintes operações de comparação numérica: $=$ (igualdade), $>$ (estritamente maior que), $>=$ (maior ou igual a), $<$ (estritamente menor que) e $<=$ (menor ou igual a).

2.5 Operações em listas e conjuntos

Para operações em conjuntos, vamos usar os símbolos clássicos das operações tradicionais como a união $\cup$, a interseção $\cap$, a diferença de conjuntos $-$ e a diferença simétrica Δ . Teste de pertinência pode ser feito com o operador **in**. Usaremos as duas barras verticais para representar o tamanho de um conjunto, como $|C|$ onde C é um conjunto.

Para operações em listas, vamos usar o que usualmente está disponível em algumas linguagens de programação. Teste de pertinência pode ser feito com o operador **in**. Usaremos as duas barras verticais para representar o tamanho de um conjunto, como $|L|$ onde L é um conjunto. O símbolo $+$ é usado para concatenar duas listas. Podemos acessar um elemento da lista da forma $L[i]$ onde L é uma lista e i é o índice do elemento, lembrando que $0 \leq i < |L|$.

3 Desvios condicionais

O desvio condicional (também chamado de estrutura de seleção ou controle de fluxo) é o mecanismo que permite ao programa tomar decisões. Sem o desvio condicional, um programa de computador seria apenas uma lista linear de instruções executadas rigidamente de cima para baixo. Com ele, o código ganha a capacidade de “pensar” e seguir caminhos diferentes dependendo da situação — exatamente como nós fazemos no dia a dia. Em pseudocódigo temos 3 opções de desvios condicionais:

- **Desvio simples** - desvio que ocorre apenas se a condição é verdadeira

```
se <condição> então
    <bloco de comandos>
fim-se
```

- **Desvio composto** - bifurcação de execução de acordo com a satisfação ou não da condição que controla o desvio

```
se <condição> então
    <bloco de comandos>
senão
    <bloco de comandos>
fim-se-senão
```

- **Múltiplos caminhos** - escolhe-se uma única variável e em cada **caso** diz-se o possível valor desta variável

```
escolha <variavel>
    caso <valor1> faça
        <bloco de comandos 1>
    caso <valor2> faça
        <bloco de comandos 2>
    caso <valor3> faça
        <bloco de comandos 3>
    ...
    caso contrário faça
        <bloco de comandos alternativo>
fim-escolha
```

4 Estruturas de repetição

Se o desvio condicional permite que o programa tome caminhos diferentes, as estruturas de repetição (também chamadas de laços, *loops* ou malhas de repetição) dão ao computador o seu maior superpoder:

a capacidade de executar o mesmo bloco de código várias vezes, de forma ultraveloz, sem que você precise reescrever as linhas.

Para que um loop não rode para sempre (o que chamamos de loop infinito, que trava o sistema), ele sempre depende de uma condição de parada. Enquanto essa condição for verdadeira, o código continua repetindo. Assim que ela se torna falsa, o programa quebra o ciclo e segue em frente.

Basicamente, existem duas formas de pensar sobre repetições:

- **Repetições definidas** - repetições que, *a priori*, sabemos quantas vezes vão executar
- **Repetições indefinidas** - repetições que não tem um valor esperado de vezes a se repetir; depende do cenário

Em pseudocódigo temos 4 opções de desvios condicionais:

- **Repetir 0 ou mais vezes** - neste caso, a estrutura de repetição testa a condição e, se for satisfeita, executa o bloco de comandos e, depois, volta à etapa de testagem da condição.

```
enquanto <condição> faça
    <bloco de comandos>
fim-enquanto
```

- **Repetir ao menos uma vez** - neste caso, a estrutura de repetição executa o bloco de comandos e testa a condição. Caso a condição seja verdadeira, o bloco de comandos é executado novamente e volta ao ponto de testes.

```
faça
    <bloco de comandos>
enquanto <condição>
```

- **Repetição por contagem** - a partícula *inc* é um valor que é somado ao contador *var* a cada volta da estrutura de repetição. Quando $inc = \pm 1$, então a parte “passo <inc>” deve ser omitida.

```
para <var> = <início> até <fim> passo <inc> faça
    <bloco de comandos>
fim-para
```

- **Iterador** - uma estrutura que permite que a variável assuma o valor de cada um dos elementos de alguma coleção, como uma lista ou um conjunto.

```
para cada <var> in <coleção>:
    <bloco de comandos>
fim-para
```

5 Reuso de código

Do ponto de vista de programação, um procedimento ou uma função é um bloco de código reaproveitável e projetado para executar uma tarefa específica. Pense nele como uma “receita de bolo” ou um *script* interno: você dá um nome a esse bloco e, sempre que precisar que aquela tarefa seja executada, você apenas chama o nome dele, em vez de reescrever todas as linhas de código novamente. A diferença essencial entre **procedimento** e **função** é que os procedimentos apenas executam tarefas e as funções retornam um valor.

Em muitas linguagens modernas (como Python, JavaScript ou C#), tudo é chamado de “função”. Nessas linguagens, um procedimento é simplesmente uma função que não tem a palavra-chave *return* (ou que retorna algo “vazio”, como *None* ou *void*).

Em pseudocódigo as declarações de funções e procedimentos devem seguir os seguintes padrões:

```
fun <nome da função>(<parâmetros>):  
    <bloco de comandos>  
    retorna <dado>  
endf  
  
proc <nome do procedimento>(<parâmetros>):  
    <bloco de comandos>  
endp
```

Vejamos um exemplo de um programinha que possui uma função:

```
fun fatorial(n : inteiro):  
    se n < 0 então  
        retorna -1  
    fim-se  
    fat ← 1  
    enquanto n > 1 faça  
        fat ← fat × n  
        n ← n - 1  
    fim-enquanto  
    retorna fat  
endf  
  
INICIO  
    ler(x)  
    f ← fatorial(x)  
    exhibir(f)  
FIM
```